

EXPERIMENT 1

Student Name: Husanpreet Kaur

Branch: AIT-CSE (Cyber Security)

Semester: 5

Subject Name: ADBMS

UID: 24BCY70138

Section/Group: 24AIT_KRG-G2

Date: 11-07-2026

Subject Code: 24CSP-333

EXPERIMENT: Advanced SQL Queries

AIM: To study and implement advanced SQL queries using conditional logic, aggregate functions, joins, self-joins, and date operations to solve real-world database problems.

Question 1: Write an SQL solution to calculate the total number of bank accounts belonging to each salary category.

The classification categories are:

1. **"Low Salary":** All monthly salaries strictly less than 20,000.
2. **"Average Salary":** All monthly salaries in the inclusive range [20,000, 50,000].
3. **"High Salary":** All monthly salaries strictly greater than 50,000.

The final result table must report all three categories, even if a category contains zero accounts.

Table:

```
CREATE TABLE Accounts (  
    account_id INT PRIMARY KEY,  
    income INT NOT NULL  
);  
INSERT INTO Accounts (account_id, income) VALUES  
(3, 108939),  
(2, 12747),  
(8, 87709),  
(6, 91738),  
(7, 45169);  
  
SELECT * FROM Accounts;
```

EXPERIMENT 1

	account_id [PK] integer	income integer
1	3	108939
2	2	12747
3	8	87709
4	6	91738
5	7	45169

Answer:

Select 'low salary' AS category, count(account_id) AS account_count from Accounts
where income < 20000
union all

Select 'average salary' AS category, count(account_id) AS account_count from
Accounts where income >= 20000 AND income <= 50000
union all

Select 'high salary' AS category, count(account_id) AS account_count from Accounts
where income > 50000;

Answer:

	category text	account_count bigint
1	low salary	1
2	average sala...	1
3	high salary	3

EXPERIMENT 1

Question 2: Design an Employee Management System database architecture by implementing the following operations sequentially:

1. Create and populate structural tracking tables for Departments, Employees, and Salaries.
2. Query the system using multi-table Joins to pull clear department breakdowns, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below \$30,000.
3. List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

Table: (i)

```
CREATE TABLE Departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    dept_id INT REFERENCES Departments(dept_id)  
);
```

```
CREATE TABLE Salaries (  
    emp_id INT PRIMARY KEY REFERENCES Employees(emp_id) ON DELETE  
CASCADE,  
    salary INT NOT NULL  
);
```

```
INSERT INTO Departments (dept_id, dept_name) VALUES  
(10, 'HR'), (20, 'IT'), (30, 'Sales');
```

```
INSERT INTO Employees (emp_id, name, dept_id) VALUES  
(1, 'Alice', 10), (2, 'Bob', 10), (3, 'Charlie', 20), (4, 'David', 20), (5, 'Emma', 30);
```

```
INSERT INTO Salaries (emp_id, salary) VALUES  
(1, 50000), (2, 25000), (3, 80000), (4, 40000), (5, 45000);
```

EXPERIMENT 1

(ii) Display Employee Details:

```
SELECT E.*, D.*, S.*
FROM
Employees AS E
JOIN
Departments AS D
ON
E.dept_id = D.dept_id
JOIN
Salaries AS S
ON
E.emp_id = S.emp_id;
```

Answer:

	emp_id integer	name character varying (50)	dept_id integer	dept_id integer	dept_name character varying (50)	emp_id integer	salary integer
1	1	Alice	10	10	HR	1	50000
2	2	Bob	10	10	HR	2	25000
3	3	Charlie	20	20	IT	3	80000
4	4	David	20	20	IT	4	40000
5	5	Emma	30	30	Sales	5	45000

(ii) Increase HR Salary by 10%:

```
UPDATE Salaries
SET salary = salary + salary * 1.10
WHERE emp_id
IN
(
SELECT E.emp_id
FROM
Employees AS E
JOIN
```

EXPERIMENT 1

```

Departments AS D
ON
E.dept_id = D.dept_id
WHERE D.dept_name = 'HR'
);

```

Select * from Salaries;

Answer:

	emp_id [PK] integer	salary integer
1	3	80000
2	4	40000
3	5	45000
4	1	5000
5	2	2500

Delete salary below 30000:

```

DELETE from Salaries
where salary<30000;

```

Answer:

	emp_id [PK] integer	salary integer
1	3	80000
2	4	40000
3	5	45000

(iii) Employees above company average salary:
SELECT

EXPERIMENT 1

```

E.emp_id,
E.name,
S.salary
FROM Employees E
JOIN Salaries S
ON E.emp_id = S.emp_id
WHERE S.salary >
(
    SELECT AVG(salary)
    FROM Salaries
);

```

Answer:

	emp_id integer	name character varying (50)	salary integer
1	3	Charlie	80000

(C)

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called pizza_toppings.

Every customer must choose **exactly 3 different toppings** to prepare a pizza.

Write a PostgreSQL query to generate **every possible 3-topping pizza combination** along with its **total ingredient cost**.

Requirements

Every pizza must contain **exactly 3 different toppings**.

The same topping **cannot appear more than once** in a pizza.

Different arrangements of the same toppings should be considered the **same pizza**.

Example:

Chicken, Pepperoni, Sausage

Pepperoni, Chicken, Sausage

Sausage, Chicken, Pepperoni

These are all the same pizza, so only **one** should appear.

Display the topping names separated by commas.

EXPERIMENT 1

Display the total ingredient cost of each pizza.

Sort the output:

First by **highest total cost**

If two pizzas have the same cost, sort them alphabetically.

Table: CREATE TABLE pizza_toppings (
 topping_name VARCHAR(50) PRIMARY KEY,
 ingredient_cost NUMERIC(4,2) NOT NULL
);

INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES
 ('Pepperoni', 0.50),
 ('Sausage', 0.70),
 ('Chicken', 0.55),
 ('Onions', 0.25),
 ('Extra Cheese', 0.40);

SELECT * FROM pizza_toppings;

	topping_name [PK] character varying (50)	ingredient_cost numeric (4,2)
1	Pepperoni	0.50
2	Sausage	0.70
3	Chicken	0.55
4	Onions	0.25
5	Extra Cheese	0.40

```
select
concat(t1.topping_name,',',t2.topping_name,',',t3.topping_name) as pizza,
(t1.ingredient_cost + t2.ingredient_cost + t3.ingredient_cost) as cost
FROM pizza_toppings as t1
join
pizza_toppings as t2
```

EXPERIMENT 1

```
on t1.topping_name<t2.topping_name
join
pizza_toppings as t3
on t2.topping_name<t3.topping_name
order by cost desc;
```

	pizza text	cost numeric
1	Chicken,Pepperoni,Sausage	1.75
2	Chicken,Extra Cheese,Sausage	1.65
3	Extra Cheese,Pepperoni,Saus...	1.60
4	Chicken,Onions,Sausage	1.50
5	Chicken,Extra Cheese,Pepper...	1.45
6	Onions,Pepperoni,Sausage	1.45
7	Extra Cheese,Onions,Sausage	1.35
8	Chicken,Onions,Pepperoni	1.30
9	Chicken,Extra Cheese,Onions	1.20
10	Extra Cheese,Onions,Pepperoni	1.15

(D) Amazon wants to identify **returning customers** who made another purchase shortly after their first purchase.

Write a PostgreSQL query to find all users who made **another purchase within 1 to 7 days** of an earlier purchase.

Requirements

1. Compare purchases **made by the same user only**.
2. Ignore comparisons where the same transaction is matched with itself.
3. The second purchase must occur **after** the first purchase.
4. Same-day purchases must **not** be considered.

EXPERIMENT 1

5. The second purchase must occur within **7 days** of the first purchase.
6. Display only the **unique User IDs** that satisfy the above conditions.
7. Sort the output in ascending order of user_id.

Table: CREATE TABLE amazon_transactions (
 id INT PRIMARY KEY,
 user_id INT NOT NULL,
 item VARCHAR(50),
 created_at TIMESTAMP NOT NULL,
 revenue INT
);

INSERT INTO amazon_transactions (id, user_id, item, created_at, revenue) VALUES
 (1, 101, 'biscuit', '2026-03-01 10:00:00', 12),
 (2, 101, 'milk', '2026-03-01 14:00:00', 5),
 (3, 101, 'bread', '2026-03-05 09:00:00', 8),
 (4, 102, 'banana', '2026-03-10 12:00:00', 4),
 (5, 102, 'apple', '2026-03-25 11:00:00', 6),
 (6, 103, 'milk', '2026-03-15 08:00:00', 5),
 (7, 103, 'bread', '2026-03-16 08:00:00', 7);

SELECT * FROM amazon_transactions;

	id [PK] integer	user_id integer	item character varying (50)	created_at timestamp without time zone	revenue integer
1	1	101	biscuit	2026-03-01 10:00:00	12
2	2	101	milk	2026-03-01 14:00:00	5
3	3	101	bread	2026-03-05 09:00:00	8
4	4	102	banana	2026-03-10 12:00:00	4
5	5	102	apple	2026-03-25 11:00:00	6
6	6	103	milk	2026-03-15 08:00:00	5
7	7	103	bread	2026-03-16 08:00:00	7

SELECT DISTINCT A.user_id

EXPERIMENT 1

```
FROM amazon_transactions A
JOIN amazon_transactions B
ON A.user_id = B.user_id
WHERE B.created_at > A.created_at
AND DATE(B.created_at) > DATE(A.created_at)
AND B.created_at <= A.created_at + INTERVAL '7 days'
ORDER BY A.user_id;
```

Answer:

	user_id integer
1	101
2	103

LEARNING OUTCOMES:

1. Learned to classify records into multiple categories using conditional filtering, calculate summary statistics with aggregate functions such as COUNT() and AVG().
2. Gained hands-on experience in creating and managing related database tables, retrieving integrated information through INNER JOIN operations, updating records with UPDATE, deleting records based on business rules using DELETE, and maintaining relationships through primary and foreign key constraints.
3. Learned to solve real-world scenarios such as generating unique combinations of records, identifying repeated customer transactions within a specified time period, and comparing rows from the same table using self-joins while eliminating duplicate and invalid comparisons.
4. Applied subqueries to compare individual values against overall database statistics, utilized aggregate functions to analyze organizational data, and performed date arithmetic and timestamp comparisons using SQL interval operations.
5. Developed the capability to translate business requirements into optimized SQL queries for domains such as banking, employee management, food ordering systems, and e-commerce,



EXPERIMENT 1

while ensuring data accuracy, consistency, and meaningful reporting through structured query design.